OLYMPUS CYBER

TOOL DOCUMENTATION

HOW-TO GUIDE

Olympus Ripper - Hybrid Forensic Artifact Parser

Document Date	May 27, 2026
Version	1.0.0
Prepared By	Olympus Cyber
Classification	CONFIDENTIAL

Having a bad day? We restore control.

Overview

Olympus Ripper (oripper) is a macOS-native Python CLI tool that parses both Windows Registry hives and macOS forensic artifacts through a unified plugin architecture. Inspired by Harlan Carvey's RegRipper, it is rebuilt in Python for modern cross-platform incident response workflows.

Key capabilities: **Windows Registry hive parsing** (SAM, SYSTEM, SOFTWARE, SECURITY, NTUSER.DAT, USRCLASS.DAT), **macOS artifact analysis** (TCC.db, KnowledgeC.db, LaunchAgents/Daemons, QuarantineEvents, FSEvents, unified logs, Safari history), **MITRE ATT&CK mapping** on every finding, and multiple output formats for SIEM integration.

Installation

Prerequisites

- Python 3.9 or later
- macOS (recommended) or Linux
- pip3 (included with Python 3)

Step 1 — Download and Extract

Download the olympus-ripper.zip archive and extract it:

```
unzip olympus-ripper.zip
cd olympus-ripper
```

Step 2 — Create a Virtual Environment

macOS 14+ blocks system-wide pip installs. Use a virtual environment:

```
python3 -m venv venv
source venv/bin/activate
```

Your terminal prompt will show (venv) when the environment is active.

Step 3 — Install Olympus Ripper

With the virtual environment active, install in editable mode:

```
pip install -e .
```

This installs all dependencies (python-registry, regipy) and creates the oripper command.

Step 4 — Verify Installation

```
oripper --version
```

Expected output: Olympus Ripper v1.0.0

Reactivating the Environment

Each new terminal session requires reactivation:

```
cd ~/olympus-ripper
source venv/bin/activate
```

Usage

Parse a Single Artifact

The rip command analyzes one file. Olympus Ripper auto-detects the file type and selects the right plugins:

```
oripper rip /path/to/SAM
oripper rip /path/to/TCC.db
oripper rip /path/to/NTUSER.DAT
```

Scan a Mounted Volume

The volume command walks a mounted forensic image and runs all applicable plugins:

```
oripper volume /Volumes/Evidence
```

This scans for both Windows Registry hives (in standard paths) and macOS artifacts.

Filter by Plugin, Platform, or Category

```
oripper rip /path/to/SYSTEM -p system_services system_network
oripper rip /path/to/NTUSER.DAT --platform windows
oripper rip /path/to/NTUSER.DAT -c persistence
```

Output Formats

Olympus Ripper supports four output formats:

Format	Flag	Use Case
Text (ANSI)	-f text	Terminal review with Olympus branding
JSON	-f json	SIEM ingest, API integration, scripted analysis
CSV	-f csv	Spreadsheet import, pivot tables
Timeline (TLN)	-f timeline	Super-timeline integration with plaso/log2timeline

Write output to a file:

```
oripper rip /path/to/SAM -f json -o findings.json
oripper rip /path/to/SOFTWARE -f timeline -o timeline.tln --hostname WORKSTATION01
```

List Available Plugins

```
oripper list
oripper list --platform macos
oripper list --category persistence
oripper list --hive-type SAM
```

Plugin Details

```
oripper info sam_users
```

Shows the plugin's description, author, version, platform, category, registry keys or default paths, and MITRE ATT&CK references.

Plugin Reference

Windows Registry Plugins

Plugin	Hive	Category	MITRE	Description
sam_users	SAM	credentials	T1003.002, T1087.001	Local user accounts, RIDs, login counts
system_services	SYSTEM	persistence	T1543.003, T1569.002	Services, drivers, auto-start entries
system_network	SYSTEM	network	T1016	Network interfaces, DHCP, DNS config
software_persistence	SOFTWARE	persistence	T1547.001, T1547.004	Run/RunOnce keys, autorun entries
software_installed	SOFTWARE	application	—	Installed applications (Uninstall keys)
software_networklist	SOFTWARE	network	T1016	Network profiles, WiFi history
ntuser_userassist	NTUSER	execution	T1059	UserAssist execution history (ROT13)
ntuser_recentdocs	NTUSER	user_activity	—	Recently opened files (MRU)
ntuser_typedpaths	NTUSER	user_activity	—	Explorer address bar history
ntuser_run	NTUSER	persistence	T1547.001	User-level Run/RunOnce keys

macOS Artifact Plugins

Plugin	Artifact	Category	MITRE	Description
macos_tcc	TCC.db	permissions	T1548, T1562.001	Camera, mic, FDA, accessibility grants
macos_knowledgec	KnowledgeC.db	user_activity	—	App usage, lock/unlock, screen time
macos_launch_persistence	LaunchAgents	persistence	T1543.001, T1543.004	Plist-based persistence mechanisms
macos_login_items	Login Items	persistence	T1547.015	Login items, startup apps, hooks
macos_quarantine	QuarantineV2	file_activity	T1566.001, T1204.002	Download history with source tracking
macos_unified_logs	Unified Logs	security	T1059, T1078	Process exec, auth, network, Gatekeeper
macos_fsevents	.fseventsd	file_activity	T1070.004	File system activity journal
macos_safari_history	History.db	user_activity	—	Safari browsing and download history

Writing Custom Plugins

Olympus Ripper auto-discovers plugins at startup. Create a .py file in any directory and point --plugin-dir at it.

macOS Plugin Template

```
from olympus_ripper.plugin_base import MacArtifactPlugin, ArtifactCategory, Finding

class MyPlugin(MacArtifactPlugin):
    name = "my_custom_plugin"
    description = "Parse a custom artifact"
    author = "Your Name"
    version = "1.0.0"
    category = ArtifactCategory.SECURITY
    artifact_type = "sqlite"
    default_paths = ["/path/to/artifact.db"]

    def run(self, target, **kwargs):
        findings = []
        findings.append(Finding(
            source="my_artifact",
            key="something_interesting",
            value="the extracted data",
            category=self.category,
            tags=["custom"],
        ))
        return findings
```

Windows Registry Plugin Template

```
from olympus_ripper.plugin_base import RegistryPlugin, ArtifactCategory, Finding

class MyRegistryPlugin(RegistryPlugin):
    name = "my_registry_plugin"
    description = "Extract custom registry data"
    author = "Your Name"
    version = "1.0.0"
    category = ArtifactCategory.PERSISTENCE
    hive_type = "SOFTWARE"
    registry_keys = ["Microsoft\\Windows\\CurrentVersion\\Run"]

    def run(self, target, **kwargs):
        # Use python-registry to parse the hive
        from Registry import Registry
        reg = Registry.Registry(target)
        findings = []
        # Your parsing logic here
        return findings
```

Running Custom Plugins

```
oripper rip /path/to/artifact -P /path/to/my/plugins/
```

The -P flag can be specified multiple times for multiple directories.

CLI Reference

Command	Syntax	Description
rip	oripper rip <target> [options]	Parse a single hive or artifact file
volume	oripper volume <path> [options]	Scan a mounted volume for all artifacts
list	oripper list [options]	List available plugins with filters
info	oripper info <plugin_name>	Show detailed plugin information

Global Flags

Flag	Description
--version	Show version and exit
--no-color	Disable ANSI colored output
-q, --quiet	Suppress progress messages
-P, --plugin-dir	Additional plugin directory (repeatable)

Rip Command Flags

Flag	Description
-p, --plugins	Specific plugin name(s) to run
-H, --hive-type	Force hive type: SAM, SYSTEM, SOFTWARE, SECURITY, NTUSER, USRCLASS
--platform	Filter plugins: windows or macos
-c, --category	Filter by category: persistence, execution, credentials, etc.
-f, --format	Output format: text, json, csv, timeline
-o, --output	Write output to file
--hostname	Hostname for timeline output (default: UNKNOWN)